

1. Setup e Imports

```
import json as _json
import json
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm import tqdm
import os
import sys
from pathlib import Path
import subprocess

# Agregar path del proyecto para imports
sys.path.append(os.path.abspath('../..'))
from sae.tools.naming_utils import save_checkpoint, get_model_name, get_extras_id, get_checkpoint_dir, get_model_dir
from sae.tools.experiment_utils import get_experiment_dir, get_plots_dir, get_metrics_dir, get_logs_dir

# Configuración de visualización para Quarto/PDF
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', None)
np.set_printoptions(linewidth=100, precision=4, suppress=True)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Dispositivo: {device}')
```

Dispositivo: cuda

2 Metadata para Naming de Checkpoints

```
# Metadata para naming de checkpoints (separada de hiperparámetros)
NAMING_META = {
    'variante': 'matryoshkabatchtopk',      # Arquitectura del SAE
    'tecnica': 'matryoshkabatchtopk',      # MatryoshkaBatchTopK
    'capa': 6,                             # Layer del modelo OthelloGPT
    'juegos': 25000,                       # Número de partidas
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt',
    'experiment_base_path': os.path.abspath('../../experiments'),
    'extras': {
        'expansion': 16,                   # d_sae / d_in = 8192 / 512
        'k': 81,                           # features activas por nivel (L0 promedio)
        'matryoshka_widths': [2048, 4096, 8192], # anchos matryoshka (último = d_sae)
        'epochs': 1000,
        'patience': 20
    }
}

print('\nMetadata de naming:')
for key, value in NAMING_META.items():
    print(f' {key}: {value}')
```

Metadata de naming:

```
variante: matryoshkabatchtopk
tecnica: matryoshkabatchtopk
capa: 6
juegos: 25000
base_path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt
experiment_base_path: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments
extras: {'expansion': 16, 'k': 81, 'matryoshka_widths': [2048, 4096, 8192], 'epochs': 1000}
```

```
extras_id = get_extras_id(
    get_checkpoint_dir(
        NAMING_META['base_path'],
        NAMING_META['variante'],
        NAMING_META['tecnica'],
        NAMING_META['capa'],
        NAMING_META['juegos']
    ),
```

```

    NAMING_META['extras']
) if NAMING_META.get('extras') else None

# get_plots_dir crea el directorio automáticamente
plots_dir = Path(get_plots_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
))

print(f'extras_id: {extras_id}')
print(f'plots_dir: {plots_dir}')

```

Nuevo extras registrado: ex1 → {'expansion': 16, 'k': 81, 'matryoshka_widths': [2048, 4096, 8192]}

extras_id: ex1

plots_dir: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_matryoshka

3. Configuración del SAE

Arquitectura MatryoshkaBatchTopK: - **Nested reconstruction losses:** se calculan pérdidas MSE en múltiples sub-anchos (`matryoshka_widths`) — cada ancho debe reconstruir el input por sí solo - **BatchTopK activation:** threshold global del batch para que el promedio de features activas sea `k` - **Sin coeficiente de sparsidad:** la sparsity se controla directamente con `k` - **Loss** = $\sum \text{MSE}(\text{recon_w}, \text{x})$ para cada ancho `w` en `matryoshka_widths`

Parámetro	L1 (standard)	BatchTopK	Matryoshka-BatchTopK
Sparsity	<code>sparsity_coef</code> (soft)	<code>k</code> (promedio del batch)	<code>k</code> (promedio, por nivel)
L0	Variable	Variable por muestra	Variable por muestra, por nivel
Loss	$\text{MSE} + \lambda \cdot \text{L1}$	MSE	$\sum \text{MSE_width}$

Las celdas siguientes entrenan el SAE directamente sobre los archivos `.npy` pre-extraídos usando la arquitectura `MatryoshkaBatchTopK`.

```

NUM_EPOCHS = NAMING_META['extras']['epochs']

cfg = {
    'architecture': 'matryoshkabatchtopk',
    'd_in': 512,
    'd_sae': 8192,
    'k': 81,
    'matryoshka_widths': [2048, 4096, 8192],
    'lr': 1e-3,
    'train_batch_size_tokens': 4096,
    'hook_name': 'blocks.5.hook_resid_post',
    # P3: betas según config.py de los autores (beta2=0.99 vs default PyTorch 0.999)
    'beta1': 0.9,
    'beta2': 0.99,
    # P1: parámetros para auxiliary loss de dead features (config.py de los autores)
    'n_batches_to_dead': 20, # batches sin activación para declarar feature muerta
    'top_k_aux': 512, # top-k features muertas usadas en L_aux
    'aux_penalty': 1/32, # peso de L_aux (paper ec. 5)
    # P2: normalización del input por muestra
    'input_unit_norm': True,
}

cfg_summary = pd.DataFrame({
    'Parámetro': ['architecture', 'd_in', 'd_sae', 'k (L0 promedio)', 'matryoshka_widths',
                  'hook_name', 'lr', 'beta1', 'beta2',
                  'n_batches_to_dead', 'top_k_aux', 'aux_penalty', 'input_unit_norm'],
    'Valor': [cfg['architecture'], cfg['d_in'], cfg['d_sae'], cfg['k'],
              str(cfg['matryoshka_widths']), cfg['hook_name'], cfg['lr'],
              cfg['beta1'], cfg['beta2'],
              cfg['n_batches_to_dead'], cfg['top_k_aux'], cfg['aux_penalty'],
              cfg['input_unit_norm']]
})

print('Configuración SAELens:')
print(cfg_summary.to_string(index=False))

```

Configuración SAELens:

Parámetro	Valor
architecture	matryoshkabatchtopk
d_in	512
d_sae	8192
k (L0 promedio)	81
matryoshka_widths	[2048, 4096, 8192]

hook_name	blocks.5.hook_resid_post
lr	0.001
beta1	0.9
beta2	0.99
n_batches_to_dead	20
top_k_aux	512
aux_penalty	0.03125
input_unit_norm	True

4. Dataset de Activaciones (Train/Validation Split 80-20%)

Activaciones pre-extraídas con `sae/activations/run_extraction.py` desde la capa `blocks.5.hook_resid_post`.

```
class ActivationsDataset(Dataset):
    """Dataset para cargar activaciones extraídas"""
    def __init__(self, activations_path):
        self.activations = np.load(activations_path, mmap_mode='r')
    def __len__(self):
        return len(self.activations)
    def __getitem__(self, idx):
        return torch.tensor(self.activations[idx], dtype=torch.float32)

activations_file = Path(f"../../activations/data/layer{NAMING_META['capa']}_{NAMING_META['hook']}")
full_dataset = ActivationsDataset(activations_file)

# Split 80-20 reproducible
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(
    full_dataset, [train_size, val_size],
    generator=torch.Generator().manual_seed(42)
)

train_loader = DataLoader(train_dataset, batch_size=cfg["train_batch_size_tokens"], shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=cfg["train_batch_size_tokens"], shuffle=False)

dataset_info = pd.DataFrame({
    'Conjunto': ['Train', 'Validación', 'Total'],
    'Muestras': [f'{len(train_dataset):,}', f'{len(val_dataset):,}', f'{len(full_dataset):,}'],
    'Shape': [str(full_dataset.activations.shape), '', ''],
})
```

```
})
print(dataset_info.to_string(index=False))
```

```

Conjunto  Muestras      Shape
Train 1,180,000 (1475000, 512)
Validación  295,000
Total 1,475,000

```

5. Modelo: MatryoshkaBatchTopK SAE

Implementación de la arquitectura MatryoshkaBatchTopK según el paper (ec. 3-5) y el código de los autores (`sae.py`).

Arquitectura: - El encoder proyecta **una sola vez** con W_{enc} completo ($d_{in} \rightarrow d_{sae}$) para obtener $f(x)$ de tamaño d_{sae} (paper ec. 3) - **BatchTopK global único:** se seleccionan los $k \times batch_size$ valores más altos sobre los $d_{sae} \times batch_size$ valores del batch completo — un único threshold compartido entre todos los anchos (paper ec. 6) - Las reconstrucciones por nivel usan **prefijos del mismo vector:** `all_acts_topk[:, 0:w]` — las features 0:2048 son literalmente las mismas en el nivel $w=2048$ y en el nivel $w=4096$ (paper ec. 4) - En **eval**, se usa un threshold persistente calibrado con EMA durante training — no se recalcula (paper ec. 7) - **L0 es variable por muestra pero promedia k en cada batch**

Parámetro	L1 (standard)	BatchTopK	Matryoshka-BatchTopK
Sparsity	<code>sparsity_coef</code> (soft)	k (promedio del batch)	k (promedio, threshold global)
L0	Variable	Variable por muestra	Variable por muestra
Loss	$MSE + \cdot L1$	MSE	$\Sigma MSE_width + \cdot L_aux$

```

class MatryoshkaBatchTopKSAE(nn.Module):
    """
    Sparse Autoencoder con activación MatryoshkaBatchTopK.
    Corregido según paper (ec. 3-5) e implementación de los autores (sae.py).

    Correcciones aplicadas:
    - P1: encoding único con  $W_{enc}$  completo; BatchTopK global; prefijos compartidos
    - P1: BatchTopK preserva valores originales ( $x * indicator$ ), threshold EMA para eval

```

```

- P1: auxiliary loss para dead features (sae.py:198-221)
- P2: make_decoder_weights_and_grad_unit_norm proyecta gradiente (sae.py:51-57)
- P2: preprocessing input_unit_norm configurable (sae.py:36-43)
- P3: W_dec inicializado como transpuesta de W_enc (sae.py:93-94)
"""
def __init__(self, d_in, d_sae, k, matryoshka_widths,
              n_batches_to_dead=20, top_k_aux=512, aux_penalty=1/32,
              input_unit_norm=False):
    super().__init__()
    self.d_in = d_in
    self.d_sae = d_sae
    self.k = k
    self.matryoshka_widths = sorted(matryoshka_widths)
    assert self.matryoshka_widths[-1] == d_sae, \
        f"El último ancho ({self.matryoshka_widths[-1]}) debe ser d_sae ({d_sae})"

    self.n_batches_to_dead = n_batches_to_dead
    self.top_k_aux = top_k_aux
    self.aux_penalty = aux_penalty
    self.input_unit_norm = input_unit_norm

    self.W_enc = nn.Parameter(torch.empty(d_in, d_sae))
    self.W_dec = nn.Parameter(torch.empty(d_sae, d_in))
    self.b_dec = nn.Parameter(torch.zeros(d_in))

    # P3: W_dec = W_enc^T normalizado (sae.py:93-94) - inicialización tied
    nn.init.kaiming_uniform_(self.W_enc)
    self.W_dec.data[:] = self.W_enc.t().data
    self.W_dec.data[:] = self.W_dec / self.W_dec.norm(dim=-1, keepdim=True)

    # P1: threshold persistente calibrado con EMA durante training (sae.py:97)
    self.register_buffer('threshold', torch.tensor(0.0))
    # P1: contador de batches sin activación por feature - para L_aux (sae.py:30-32)
    self.register_buffer('num_batches_not_active', torch.zeros(d_sae))

# -----
# P2: Preprocessing del input (sae.py:36-48)
# -----

def preprocess_input(self, x):
    if self.input_unit_norm:
        x_mean = x.mean(dim=-1, keepdim=True)

```

```

        x = x - x_mean
        x_std = x.std(dim=-1, keepdim=True)
        x = x / (x_std + 1e-5)
        return x, x_mean, x_std
    return x, None, None

def postprocess_output(self, x_reconstruct, x_mean, x_std):
    if self.input_unit_norm and x_mean is not None:
        x_reconstruct = x_reconstruct * x_std + x_mean
    return x_reconstruct

# -----
# P1: Encoding único + BatchTopK global (sae.py:100-119)
# -----

def compute_activations(self, x_cent):
    # P1: encoding ÚNICO con W_enc completo - un solo pase para todos los anchos
    # Las features 0:w son literalmente las mismas en todos los niveles (paper ec. 3)
    pre_acts = x_cent @ self.W_enc      # [batch, d_sae]
    acts      = F.relu(pre_acts)        # ReLU antes del topk (sae.py:102)

    if self.training:
        # P1: BatchTopK global sobre d_sae*batch_size valores (paper ec. 6)
        # Preserva valores originales con scatter - NO resta threshold (sae.py:104-114)
        topk_result = torch.topk(acts.flatten(), self.k * x_cent.shape[0])
        acts_topk = (
            torch.zeros_like(acts.flatten())
            .scatter(-1, topk_result.indices, topk_result.values)
            .reshape(acts.shape)
        )
        self._update_threshold(acts_topk)
    else:
        # P1: eval usa threshold persistente, no recalcula (paper ec. 7, sae.py:117)
        acts_topk = torch.where(acts > self.threshold, acts, torch.zeros_like(acts))

    return acts, acts_topk

@torch.no_grad()
def _update_threshold(self, acts_topk, lr=0.01):
    # P1: EMA del mínimo valor positivo - calibra para eval (sae.py:224-228)
    positive_mask = acts_topk > 0
    if positive_mask.any():

```



```

        min_positive = acts_topk[positive_mask].min()
        self.threshold = (1 - lr) * self.threshold + lr * min_positive

@torch.no_grad()
def _update_inactive_features(self, acts_topk):
    # P1: necesario para L_aux - trackea qué features no se activan (sae.py:60-62)
    self.num_batches_not_active += (acts_topk.sum(0) == 0).float()
    self.num_batches_not_active[acts_topk.sum(0) > 0] = 0

# -----
# Forward
# -----

def forward(self, x):
    """
    Retorna:
        outputs      : lista de (recon_w, hidden_w) para cada ancho matryoshka
                       recon_w está en espacio normalizado (= espacio original si input_unit_norm=True)
        all_acts      : pre-topk [batch, d_sae] - para L_aux
        all_acts_topk: post-topk [batch, d_sae]
        x_proc        : x preprocesado (= x si input_unit_norm=False)
    """
    x_proc, x_mean, x_std = self.preprocess_input(x)
    x_cent = x_proc - self.b_dec
    all_acts, all_acts_topk = self.compute_activations(x_cent)

    # P1: reconstrucciones por PREFIJO compartido (paper ec. 4, sae.py:149-155)
    # all_acts_topk[:, 0:w] son las mismas features en todos los niveles anidados
    # recon_w se mantiene en espacio normalizado para que la loss compare espacios iguales
    outputs = []
    for w in self.matryoshka_widths:
        hidden_w = all_acts_topk[:, :w]
        recon_w = hidden_w @ self.W_dec[:, :w] + self.b_dec
        outputs.append((recon_w, hidden_w))

    self._update_inactive_features(all_acts_topk)
    return outputs, all_acts, all_acts_topk, x_proc

# -----
# P1: Auxiliary loss para dead features (sae.py:198-221)
# -----

```

```

def get_auxiliary_loss(self, x_proc, x_reconstruct_full, all_acts):
    residual = x_proc.float() - x_reconstruct_full.float()
    aux_reconstruct = torch.zeros_like(residual)

    dead_features = self.num_batches_not_active >= self.n_batches_to_dead

    if dead_features.sum() > 0:
        acts_topk_aux = torch.topk(
            all_acts[:, dead_features],
            min(self.top_k_aux, int(dead_features.sum()))),
            dim=-1,
        )
        acts_aux = torch.zeros_like(all_acts[:, dead_features]).scatter(
            -1, acts_topk_aux.indices, acts_topk_aux.values
        )
        aux_reconstruct = aux_reconstruct + acts_aux @ self.W_dec[dead_features]

    if aux_reconstruct.abs().sum() > 0:
        return self.aux_penalty * (aux_reconstruct.float() - residual.float()).pow(2).mean()
    return torch.tensor(0.0, device=x_proc.device)

# -----
# P2: Constraint de norma unitaria con proyección del gradiente (sae.py:51-57)
# -----

@torch.no_grad()
def make_decoder_weights_and_grad_unit_norm(self):
    # P2: proyecta gradiente al espacio tangente de la esfera unitaria + normaliza
    # @torch.no_grad() evita trazar grafo interno - no impide leer .grad (sae.py:50)
    # Llamar ANTES de optimizer.step() (training.py:31)
    W_dec_normed = self.W_dec / self.W_dec.norm(dim=-1, keepdim=True)
    W_dec_grad_proj = (
        (self.W_dec.grad * W_dec_normed).sum(-1, keepdim=True) * W_dec_normed
    )
    self.W_dec.grad -= W_dec_grad_proj
    self.W_dec.data = W_dec_normed

# Crear modelo
model = MatryoshkaBatchTopKSAE(
    d_in=cfg['d_in'],
    d_sae=cfg['d_sae'],

```

```

k=cfg['k'],
matryoshka_widths=cfg['matryoshka_widths'],
n_batches_to_dead=cfg['n_batches_to_dead'],
top_k_aux=cfg['top_k_aux'],
aux_penalty=cfg['aux_penalty'],
input_unit_norm=cfg['input_unit_norm'],
).to(device)
num_params = sum(p.numel() for p in model.parameters())

model_info = pd.DataFrame({
    'Componente': [
        'Input', 'Encoder (W_enc)', 'Activación', 'Anchos matryoshka',
        'Hidden activos (por nivel)', 'Decoder (W_dec)', 'Output', 'Total params'
    ],
    'Dimensión': [
        f'{cfg["d_in"]}',
        f'{cfg["d_in"]} -> {cfg["d_sae"]} (único pase, prefijos compartidos)',
        f'BatchTopK global (k={cfg["k"]}) - threshold EMA en eval',
        str(cfg['matryoshka_widths']),
        f' {cfg["k"]} activos (promedio del batch)',
        f'{cfg["d_sae"]} -> {cfg["d_in"]} (slices de prefijo)',
        f'{cfg["d_in"]}',
        f'{num_params:,}'
    ]
})
print(model_info.to_string(index=False))

```

Componente	Dimensión
Input	512
Encoder (W_enc)	512 -> 8192 (único pase, prefijos compartidos)
Activación	BatchTopK global (k=81) - threshold EMA en eval
Anchos matryoshka	[2048, 4096, 8192]
Hidden activos (por nivel)	81 activos (promedio del batch)
Decoder (W_dec)	8192 -> 512 (slices de prefijo)
Output	512
Total params	8,389,120

6. Función de Pérdida MatryoshkaBatchTopK

Loss = mean MSE(recon_w, x) para cada ancho w matryoshka_widths +
 · L_aux

- **MSE en cada nivel:** error de reconstrucción desde cada sub-espacio matryoshka
- **Sin L1 penalty:** la sparsity está controlada por el threshold global de BatchTopK
- **Loss total = media de MSE en todos los anchos** — mantiene la escala estable y preserva el balance entre reconstrucción y aux_loss independientemente del número de anchos (alineado con Bussmann sae.py:172)
- La loss anidada fuerza a que los sub-espacios más pequeños también sean representaciones completas del input

```
def loss_function(x_proc, outputs, all_acts, model):
    """
    Pérdida MatryoshkaBatchTopK: media de MSE en cada ancho + auxiliary loss.

    Paper ec. 5:  $L(x) = \sum_{m \in M} \|x - \hat{x}\|^2 + \cdot L_{aux}$ 
    Bussmann (sae.py): promedia dividiendo por n_grupos - mantiene la escala de
    la MSE estable independientemente del número de anchos, y preserva el balance
    entre reconstrucción y aux_loss.

    Args:
        x_proc: activaciones preprocesadas [batch, d_in] (= x si input_unit_norm=False)
        outputs: lista de (recon_w, hidden_w) por ancho matryoshka
        all_acts: activaciones pre-topk [batch, d_sae] - para L_aux
        model: referencia al modelo (para get_auxiliary_loss)

    Returns:
        total_loss, mse_per_width, aux_loss
    """
    x_reconstruct_full = outputs[-1][0] # reconstrucción del ancho máximo

    # Media de MSE en todos los anchos (alineado con Bussmann sae.py:172)
    mse_per_width = [F.mse_loss(recon_w, x_proc) for recon_w, _ in outputs]
    mean_mse = sum(mse_per_width) / len(mse_per_width)

    # P1: auxiliary loss para dead features (paper ec. 5:  $\cdot L_{aux}$ , sae.py:198-221)
    aux_loss = model.get_auxiliary_loss(x_proc, x_reconstruct_full, all_acts)

    return mean_mse + aux_loss, mse_per_width, aux_loss

# P3: betas según config.py de los autores - beta2=0.99 vs default PyTorch 0.999
# (config.py: beta1=0.9, beta2=0.99)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=cfg["lr"],
    betas=(cfg["beta1"], cfg["beta2"]))
```

```

)

optim_info = pd.DataFrame({
    'Parámetro': ['Optimizador', 'lr', 'beta1', 'beta2',
                  'k (L0 promedio)', 'Anchos matryoshka',
                  'aux_penalty', 'n_batches_to_dead', 'top_k_aux'],
    'Valor':      ['Adam', cfg["lr"], cfg["beta1"], cfg["beta2"],
                  cfg["k"], str(cfg['matryoshka_widths']),
                  cfg['aux_penalty'], cfg['n_batches_to_dead'], cfg['top_k_aux']]
})
print(optim_info.to_string(index=False))

```

Parámetro	Valor
Optimizador	Adam
lr	0.001
beta1	0.9
beta2	0.99
k (L0 promedio)	81
Anchos matryoshka	[2048, 4096, 8192]
aux_penalty	0.03125
n_batches_to_dead	20
top_k_aux	512

7. Loop de Entrenamiento

```

PATIENCE = NAMING_META['extras']['patience']
WIDTHS    = cfg['matryoshka_widths']

save_dir = Path('./saved_model')
save_dir.mkdir(exist_ok=True)
best_model_name = get_model_name(
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='best',
    extras_id=extras_id
)
BEST_CKPT = save_dir / best_model_name

```

```

history = {
    'train_total_loss': [],
    'train_aux_loss': [],
    'train_l0_sparsity': [],
    'train_fraction_active': [],
    'val_total_loss': [],
    'val_aux_loss': [],
    'val_l0_sparsity': [],
    'val_fraction_active': [],
}
for w in WIDTHS:
    history[f'val_mse_w{w}'] = []

best_val_loss = float('inf')
best_epoch = 0
epochs_no_improve = 0

print(f'Iniciando entrenamiento: {NUM_EPOCHS} épocas máximo')
print(f'Early stopping: patience={PATIENCE}')
print(f'k = {cfg["k"]} features activas (L0 promedio k por batch)')
print(f'Anchos matryoshka: {WIDTHS}')
print(f'Best checkpoint: {BEST_CKPT}')
print('=' * 70)

for epoch in range(NUM_EPOCHS):

    # ----- TRAIN
    model.train()
    t_total = t_aux = t_l0 = t_frac = 0.0

    for batch in train_loader:
        x = batch.to(device)
        optimizer.zero_grad()

        # P1: forward devuelve (outputs, all_acts, all_acts_topk, x_proc)
        outputs, all_acts, all_acts_topk, x_proc = model(x)
        total_loss, _, aux_loss_val = loss_function(x_proc, outputs, all_acts, model)
        total_loss.backward()

        # P2: proyectar gradiente al espacio tangente de la esfera + normalizar
        # ANTES de optimizer.step() (sae.py:51-57, training.py:31)
        model.make_decoder_weights_and_grad_unit_norm()

```

```

optimizer.step()
# REMOVED: model.normalize_decoder() - reemplazado por make_decoder_weights_and_grad

with torch.no_grad():
    hidden_full = outputs[-1][1]
    t_total += total_loss.item()
    t_aux    += aux_loss_val.item()
    t_l0     += (hidden_full > 0).float().sum(dim=1).mean().item()
    t_frac   += (hidden_full > 0).float().mean().item()

n = len(train_loader)
history['train_total_loss'].append(t_total / n)
history['train_aux_loss'].append(t_aux / n)
history['train_l0_sparsity'].append(t_l0 / n)
history['train_fraction_active'].append(t_frac / n)

# ----- VALIDATION
model.eval()
v_total = v_aux = v_l0 = v_frac = 0.0
v_per_width = {w: 0.0 for w in WIDTHHS}

with torch.no_grad():
    for batch in val_loader:
        x = batch.to(device)
        outputs, all_acts, all_acts_topk, x_proc = model(x)
        total_loss, mse_per_width, aux_loss_val = loss_function(x_proc, outputs, all_acts)
        hidden_full = outputs[-1][1]

        v_total += total_loss.item()
        v_aux    += aux_loss_val.item()
        v_l0     += (hidden_full > 0).float().sum(dim=1).mean().item()
        v_frac   += (hidden_full > 0).float().mean().item()
        for w, mse_w in zip(WIDTHHS, mse_per_width):
            v_per_width[w] += mse_w.item()

nv = len(val_loader)
history['val_total_loss'].append(v_total / nv)
history['val_aux_loss'].append(v_aux / nv)
history['val_l0_sparsity'].append(v_l0 / nv)
history['val_fraction_active'].append(v_frac / nv)
for w in WIDTHHS:
    history[f'val_mse_w{w}'].append(v_per_width[w] / nv)

```

```

width_str = ' | '.join(f'w{w}={history[f"val_mse_w{w}"][-1]:.4f}' for w in WIDTHS)
print(f"Época {epoch+1:3d}/{NUM_EPOCHS} | "
      f"Train: {history['train_total_loss'][-1]:.6f} | "
      f"Val: {history['val_total_loss'][-1]:.6f} | "
      f"L_aux: {history['val_aux_loss'][-1]:.6f} | "
      f"L0: {history['val_l0_sparsity'][-1]:.1f} | "
      f"{width_str}")

# Early stopping (basado en loss total de validación)
if history['val_total_loss'][-1] < best_val_loss - 1e-4:
    best_val_loss = history['val_total_loss'][-1]
    best_epoch = epoch + 1
    epochs_no_improve = 0
    torch.save(model.state_dict(), BEST_CKPT)
else:
    epochs_no_improve += 1
    if epochs_no_improve >= PATIENCE:
        print(f'\nEarly stopping en época {epoch+1} (mejor: época {best_epoch})')
        break

# Restaurar mejor modelo
model.load_state_dict(torch.load(BEST_CKPT, weights_only=True))
print(f'\nMejor época: {best_epoch} | Mejor Val Loss: {best_val_loss:.6f}')

```

Iniciando entrenamiento: 1000 épocas máximo

Early stopping: patience=20

k = 81 features activas (L0 promedio k por batch)

Anchos matryoshka: [2048, 4096, 8192]

Best checkpoint: saved_model\sae_matryoshkabatchtopk_matryoshkabatchtopk_l6_25000g_ex1_best.

=====

Época	1/1000		Train: 0.325417		Val: 0.263034		L_aux: 0.000108		L0: 88.3		w2048=0.2792
Época	2/1000		Train: 0.259799		Val: 0.258514		L_aux: 0.007518		L0: 77.2		w2048=0.2654
Época	3/1000		Train: 0.243369		Val: 0.240909		L_aux: 0.006992		L0: 79.2		w2048=0.2480
Época	4/1000		Train: 0.231340		Val: 0.229609		L_aux: 0.006661		L0: 80.6		w2048=0.2363
Época	5/1000		Train: 0.222280		Val: 0.223333		L_aux: 0.006479		L0: 80.9		w2048=0.2297
Época	6/1000		Train: 0.216820		Val: 0.219897		L_aux: 0.006375		L0: 80.6		w2048=0.2262
Época	7/1000		Train: 0.214197		Val: 0.216180		L_aux: 0.006271		L0: 80.9		w2048=0.2221
Época	8/1000		Train: 0.209488		Val: 0.213511		L_aux: 0.005692		L0: 80.5		w2048=0.2200
Época	9/1000		Train: 0.208268		Val: 0.211766		L_aux: 0.006054		L0: 80.5		w2048=0.2178
Época	10/1000		Train: 0.207013		Val: 0.209665		L_aux: 0.006074		L0: 80.8		w2048=0.2156
Época	11/1000		Train: 0.205303		Val: 0.207158		L_aux: 0.006006		L0: 81.5		w2048=0.2127

Época	12/1000		Train:	0.204033		Val:	0.206737		L_aux:	0.005994		L0:	81.0		w2048=0.2122
Época	13/1000		Train:	0.202930		Val:	0.206476		L_aux:	0.005988		L0:	80.4		w2048=0.2118
Época	14/1000		Train:	0.201932		Val:	0.204355		L_aux:	0.005930		L0:	81.3		w2048=0.2094
Época	15/1000		Train:	0.200831		Val:	0.204661		L_aux:	0.005940		L0:	80.5		w2048=0.2096
Época	16/1000		Train:	0.200103		Val:	0.203314		L_aux:	0.005904		L0:	81.0		w2048=0.2080
Época	17/1000		Train:	0.199384		Val:	0.203369		L_aux:	0.005908		L0:	80.5		w2048=0.2080
Época	18/1000		Train:	0.198986		Val:	0.203333		L_aux:	0.005829		L0:	80.2		w2048=0.2079
Época	19/1000		Train:	0.197941		Val:	0.202793		L_aux:	0.005895		L0:	80.3		w2048=0.2071
Época	20/1000		Train:	0.198091		Val:	0.202287		L_aux:	0.005882		L0:	80.4		w2048=0.2065
Época	21/1000		Train:	0.197675		Val:	0.201350		L_aux:	0.005857		L0:	80.8		w2048=0.2054
Época	22/1000		Train:	0.197198		Val:	0.200740		L_aux:	0.005841		L0:	80.9		w2048=0.2047
Época	23/1000		Train:	0.196974		Val:	0.201270		L_aux:	0.005857		L0:	80.4		w2048=0.2052
Época	24/1000		Train:	0.196540		Val:	0.200131		L_aux:	0.005827		L0:	80.9		w2048=0.2039
Época	25/1000		Train:	0.196150		Val:	0.199998		L_aux:	0.005824		L0:	80.9		w2048=0.2037
Época	26/1000		Train:	0.195826		Val:	0.199697		L_aux:	0.005817		L0:	80.9		w2048=0.2032
Época	27/1000		Train:	0.195040		Val:	0.200575		L_aux:	0.005843		L0:	80.2		w2048=0.2041
Época	28/1000		Train:	0.195562		Val:	0.199747		L_aux:	0.005820		L0:	80.6		w2048=0.2032
Época	29/1000		Train:	0.195419		Val:	0.199318		L_aux:	0.005730		L0:	80.7		w2048=0.2028
Época	30/1000		Train:	0.195103		Val:	0.198601		L_aux:	0.005789		L0:	81.1		w2048=0.2019
Época	31/1000		Train:	0.194851		Val:	0.199618		L_aux:	0.005741		L0:	80.3		w2048=0.2030
Época	32/1000		Train:	0.194643		Val:	0.198174		L_aux:	0.005702		L0:	81.1		w2048=0.2014
Época	33/1000		Train:	0.194515		Val:	0.198506		L_aux:	0.005788		L0:	80.9		w2048=0.2017
Época	34/1000		Train:	0.194238		Val:	0.198623		L_aux:	0.005792		L0:	80.7		w2048=0.2018
Época	35/1000		Train:	0.194117		Val:	0.198519		L_aux:	0.005790		L0:	80.7		w2048=0.2016
Época	36/1000		Train:	0.194107		Val:	0.198640		L_aux:	0.005793		L0:	80.5		w2048=0.2017
Época	37/1000		Train:	0.194013		Val:	0.198851		L_aux:	0.005801		L0:	80.4		w2048=0.2019
Época	38/1000		Train:	0.193923		Val:	0.198196		L_aux:	0.005783		L0:	80.7		w2048=0.2012
Época	39/1000		Train:	0.193654		Val:	0.198015		L_aux:	0.005779		L0:	80.7		w2048=0.2009
Época	40/1000		Train:	0.193739		Val:	0.197165		L_aux:	0.005755		L0:	81.2		w2048=0.2000
Época	41/1000		Train:	0.193356		Val:	0.197648		L_aux:	0.005769		L0:	80.8		w2048=0.2005
Época	42/1000		Train:	0.193482		Val:	0.197494		L_aux:	0.005766		L0:	80.8		w2048=0.2003
Época	43/1000		Train:	0.193296		Val:	0.198748		L_aux:	0.005802		L0:	80.1		w2048=0.2015
Época	44/1000		Train:	0.193286		Val:	0.198014		L_aux:	0.005782		L0:	80.5		w2048=0.2007
Época	45/1000		Train:	0.193111		Val:	0.197985		L_aux:	0.005782		L0:	80.4		w2048=0.2007
Época	46/1000		Train:	0.193059		Val:	0.197359		L_aux:	0.005764		L0:	80.8		w2048=0.2000
Época	47/1000		Train:	0.192912		Val:	0.197534		L_aux:	0.005770		L0:	80.6		w2048=0.2002
Época	48/1000		Train:	0.192909		Val:	0.196417		L_aux:	0.005739		L0:	81.3		w2048=0.1990
Época	49/1000		Train:	0.192863		Val:	0.197501		L_aux:	0.005769		L0:	80.5		w2048=0.2001
Época	50/1000		Train:	0.192782		Val:	0.197177		L_aux:	0.005761		L0:	80.7		w2048=0.1997
Época	51/1000		Train:	0.192618		Val:	0.197383		L_aux:	0.005767		L0:	80.6		w2048=0.1999
Época	52/1000		Train:	0.192710		Val:	0.196962		L_aux:	0.005755		L0:	80.8		w2048=0.1994
Época	53/1000		Train:	0.192499		Val:	0.197560		L_aux:	0.005774		L0:	80.4		w2048=0.2000
Época	54/1000		Train:	0.192501		Val:	0.196464		L_aux:	0.005742		L0:	81.0		w2048=0.1989

Época	55/1000		Train:	0.192341		Val:	0.197789		L_aux:	0.005781		L0:	80.2		w2048=0.2002
Época	56/1000		Train:	0.192368		Val:	0.197288		L_aux:	0.005766		L0:	80.5		w2048=0.1997
Época	57/1000		Train:	0.192320		Val:	0.198210		L_aux:	0.005793		L0:	79.9		w2048=0.2006
Época	58/1000		Train:	0.192297		Val:	0.196098		L_aux:	0.005733		L0:	81.1		w2048=0.1984
Época	59/1000		Train:	0.192285		Val:	0.196608		L_aux:	0.005670		L0:	80.7		w2048=0.1990
Época	60/1000		Train:	0.192331		Val:	0.196199		L_aux:	0.005736		L0:	81.0		w2048=0.1985
Época	61/1000		Train:	0.192146		Val:	0.196022		L_aux:	0.005732		L0:	81.1		w2048=0.1983
Época	62/1000		Train:	0.192094		Val:	0.196614		L_aux:	0.005750		L0:	80.7		w2048=0.1988
Época	63/1000		Train:	0.192056		Val:	0.196909		L_aux:	0.005757		L0:	80.5		w2048=0.1992
Época	64/1000		Train:	0.191938		Val:	0.197072		L_aux:	0.005763		L0:	80.4		w2048=0.1993
Época	65/1000		Train:	0.192034		Val:	0.196181		L_aux:	0.005738		L0:	80.9		w2048=0.1984
Época	66/1000		Train:	0.191954		Val:	0.196863		L_aux:	0.005678		L0:	80.5		w2048=0.1992
Época	67/1000		Train:	0.191831		Val:	0.196728		L_aux:	0.005754		L0:	80.6		w2048=0.1989
Época	68/1000		Train:	0.191772		Val:	0.196742		L_aux:	0.005754		L0:	80.5		w2048=0.1989
Época	69/1000		Train:	0.191826		Val:	0.196983		L_aux:	0.005762		L0:	80.4		w2048=0.1992
Época	70/1000		Train:	0.191845		Val:	0.197258		L_aux:	0.005692		L0:	80.1		w2048=0.1995
Época	71/1000		Train:	0.191665		Val:	0.195986		L_aux:	0.005655		L0:	80.9		w2048=0.1982
Época	72/1000		Train:	0.191732		Val:	0.195642		L_aux:	0.005725		L0:	81.1		w2048=0.1977
Época	73/1000		Train:	0.191741		Val:	0.196885		L_aux:	0.005761		L0:	80.4		w2048=0.1989
Época	74/1000		Train:	0.191357		Val:	0.195958		L_aux:	0.005735		L0:	80.9		w2048=0.1980
Época	75/1000		Train:	0.191652		Val:	0.196623		L_aux:	0.005754		L0:	80.5		w2048=0.1987
Época	76/1000		Train:	0.191571		Val:	0.196997		L_aux:	0.005686		L0:	80.2		w2048=0.1991
Época	77/1000		Train:	0.191564		Val:	0.196327		L_aux:	0.005745		L0:	80.6		w2048=0.1983
Época	78/1000		Train:	0.191550		Val:	0.196586		L_aux:	0.005753		L0:	80.5		w2048=0.1986
Época	79/1000		Train:	0.191574		Val:	0.196274		L_aux:	0.005745		L0:	80.6		w2048=0.1982
Época	80/1000		Train:	0.191437		Val:	0.196883		L_aux:	0.005763		L0:	80.3		w2048=0.1988
Época	81/1000		Train:	0.191450		Val:	0.196753		L_aux:	0.005681		L0:	80.3		w2048=0.1988
Época	82/1000		Train:	0.191499		Val:	0.195627		L_aux:	0.005726		L0:	81.0		w2048=0.1976
Época	83/1000		Train:	0.191451		Val:	0.195446		L_aux:	0.005721		L0:	81.1		w2048=0.1974
Época	84/1000		Train:	0.191406		Val:	0.195842		L_aux:	0.005655		L0:	80.8		w2048=0.1979
Época	85/1000		Train:	0.191437		Val:	0.196107		L_aux:	0.005741		L0:	80.7		w2048=0.1980
Época	86/1000		Train:	0.191310		Val:	0.196166		L_aux:	0.005744		L0:	80.6		w2048=0.1980
Época	87/1000		Train:	0.191242		Val:	0.196846		L_aux:	0.005763		L0:	80.2		w2048=0.1987
Época	88/1000		Train:	0.191329		Val:	0.195722		L_aux:	0.005652		L0:	80.8		w2048=0.1977
Época	89/1000		Train:	0.191305		Val:	0.196005		L_aux:	0.005662		L0:	80.7		w2048=0.1979
Época	90/1000		Train:	0.191252		Val:	0.196772		L_aux:	0.005683		L0:	80.2		w2048=0.1987
Época	91/1000		Train:	0.191214		Val:	0.195737		L_aux:	0.005731		L0:	80.8		w2048=0.1976
Época	92/1000		Train:	0.191213		Val:	0.196375		L_aux:	0.005672		L0:	80.4		w2048=0.1983
Época	93/1000		Train:	0.191195		Val:	0.195976		L_aux:	0.005661		L0:	80.7		w2048=0.1979
Época	94/1000		Train:	0.191022		Val:	0.196033		L_aux:	0.005741		L0:	80.6		w2048=0.1978
Época	95/1000		Train:	0.191218		Val:	0.195646		L_aux:	0.005652		L0:	80.9		w2048=0.1975
Época	96/1000		Train:	0.191092		Val:	0.195906		L_aux:	0.005738		L0:	80.6		w2048=0.1977
Época	97/1000		Train:	0.191129		Val:	0.196055		L_aux:	0.005742		L0:	80.6		w2048=0.1978

Época	98/1000		Train:	0.191054		Val:	0.196034		L_aux:	0.005664		L0:	80.5		w2048=0.1979
Época	99/1000		Train:	0.191142		Val:	0.195569		L_aux:	0.005728		L0:	80.9		w2048=0.1973
Época	100/1000		Train:	0.191036		Val:	0.197305		L_aux:	0.005778		L0:	79.8		w2048=0.1991
Época	101/1000		Train:	0.191039		Val:	0.195910		L_aux:	0.005739		L0:	80.6		w2048=0.1976
Época	102/1000		Train:	0.191063		Val:	0.195603		L_aux:	0.005730		L0:	80.8		w2048=0.1973
Época	103/1000		Train:	0.191045		Val:	0.195853		L_aux:	0.005737		L0:	80.7		w2048=0.1976

Early stopping en época 103 (mejor: época 83)

Mejor época: 83 | Mejor Val Loss: 0.195446

8. Guardar Modelo

```
# Guardar checkpoint final
checkpoint = {
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'config': {
        'architecture': cfg['architecture'],
        'd_in': cfg['d_in'],
        'd_sae': cfg['d_sae'],
        'k': cfg['k'],
        'matryoshka_widths': cfg['matryoshka_widths'],
        'lr': cfg['lr'],
        'hook_name': cfg['hook_name'],
    },
    'history': history,
}

final_model_name = get_model_name(
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='final',
    extras_id=extras_id
)

checkpoint_path = save_dir / final_model_name
torch.save(checkpoint, checkpoint_path)
print(f'Modelo guardado: {checkpoint_path}')
```

```

save_checkpoint(
    model,
    base_path=NAMING_META['base_path'],
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='final',
    extras=NAMING_META['extras'],
    config=cfg,
    history=history,
    optimizer=optimizer
)

```

Modelo guardado: saved_model\sae_matryoshkabatchtopk_matryoshkabatchtopk_l6_25000g_ex1_final

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

WindowsPath('C:/Users/Esposa/Documents/Repos/sae-othello-gpt/layer_06/models/sae_matryoshkab

9. Métricas básicas del modelo

```

metrics_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

training_metrics = {
    # Calidad de reconstrucción (loss total = media de MSE en todos los anchos)
    'val_total_loss_final': history['val_total_loss'][-1],
    'val_total_loss_best': best_val_loss,
    # MSE por ancho matryoshka (último epoch)
    **{f'val_mse_w{w}_final': history[f'val_mse_w{w}'][-1] for w in WIDTHS},
    # Sparsity (L0 del ancho máximo)
    'val_l0_final': history['val_l0_sparsity'][-1],
    'val_fraction_active_final': history['val_fraction_active'][-1],
    # Contexto del entrenamiento
}

```

```

    'best_epoch':          best_epoch,
    'total_epochs':       len(history['train_total_loss']),
    'train_total_loss_final': history['train_total_loss'][-1],
    'train_l0_final':     history['train_l0_sparsity'][-1],
    # Hiperparámetros MatryoshkaBatchTopK
    'architecture':      cfg['architecture'],
    'k':                  cfg['k'],
    'matryoshka_widths':  cfg['matryoshka_widths'],
    'd_sae':              cfg['d_sae'],
    'd_in':               cfg['d_in'],
    'expansion_factor':   cfg['d_sae'] // cfg['d_in'],
    'hook_name':          cfg['hook_name'],
}

os.makedirs(metrics_dir, exist_ok=True)
metrics_path = Path(metrics_dir) / 'training_metrics.json'
with open(metrics_path, 'w') as f:
    json.dump(training_metrics, f, indent=2)

print(f'Métricas guardadas: {metrics_path}')
print(pd.DataFrame(
    list(training_metrics.items()), columns=['Métrica', 'Valor']
).to_string(index=False))

```

Métricas guardadas: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\s

Métrica	Valor
val_total_loss_final	0.195853
val_total_loss_best	0.195446
val_mse_w2048_final	0.197566
val_mse_w4096_final	0.189179
val_mse_w8192_final	0.183601
val_l0_final	80.691922
val_fraction_active_final	0.00985
best_epoch	83
total_epochs	103
train_total_loss_final	0.191045
train_l0_final	81.0
architecture	matryoshkabatchtopk
k	81
matryoshka_widths	[2048, 4096, 8192]
d_sae	8192
d_in	512

```
expansion_factor          16
hook_name blocks.5.hook_resid_post
```

10. Visualización de Resultados

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Total Nested Loss
axes[0].plot(history['train_total_loss'], label='Train', color='steelblue', alpha=0.8)
axes[0].plot(history['val_total_loss'], label='Validación', color='tomato', alpha=0.8)
if best_epoch > 0:
    axes[0].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7,
                    label=f'Best (época {best_epoch})')
axes[0].set_title(f'Total Nested Loss ( $\Sigma$  MSE en {len(WIDTHS)} anchos)')
axes[0].set_xlabel('Época')
axes[0].set_ylabel('Loss total')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# L0 sparsity (ancho máximo, L0 promedio k)
axes[1].plot(history['train_l0_sparsity'], label='Train', color='steelblue', alpha=0.8)
axes[1].plot(history['val_l0_sparsity'], label='Validación', color='tomato', alpha=0.8)
axes[1].axhline(cfg["k"], color='orange', linestyle='--', alpha=0.8,
                label=f'k={cfg["k"]} (promedio objetivo)')
axes[1].set_title(f'L0 Sparsity - ancho máximo (w={cfg["d_sae"]}, promedio k={cfg["k"]})')
axes[1].set_xlabel('Época')
axes[1].set_ylabel('Features activas (promedio)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# MSE por ancho matryoshka (validación)
colors_w = plt.cm.viridis([i / len(WIDTHS) for i in range(len(WIDTHS))])
for w, color in zip(WIDTHS, colors_w):
    axes[2].plot(history[f'val_mse_w{w}'], label=f'w={w}', color=color, alpha=0.85)
if best_epoch > 0:
    axes[2].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7,
                    label=f'Best (época {best_epoch})')
axes[2].set_title('Val MSE por Ancho Matryoshka')
axes[2].set_xlabel('Época')
axes[2].set_ylabel('MSE')
axes[2].legend()
```

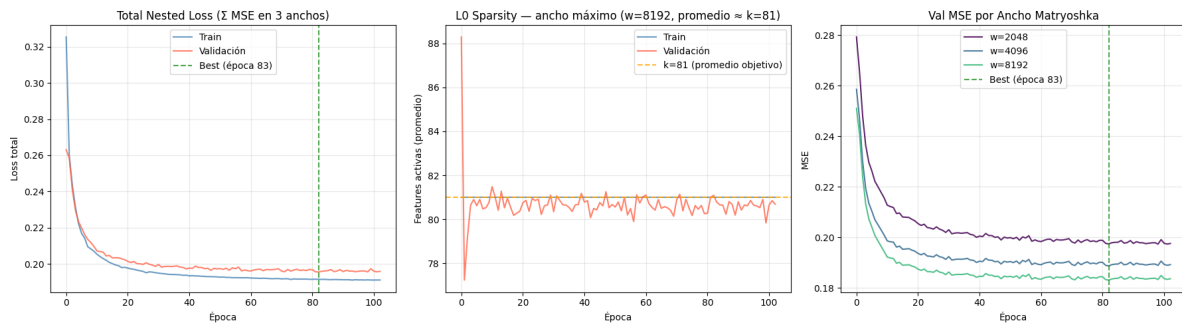
```

axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(plots_dir / 'training_curves.png', dpi=300, bbox_inches='tight')
plt.show()

print('\n' + '='*50)
print('Gráficas guardadas exitosamente')
print('='*50)

```



```

=====
Gráficas guardadas exitosamente
=====

```

12. Generar PDF con Quarto

Una vez ejecutado todo el notebook, ejecutar el siguiente comando en la terminal para generar el PDF con Quarto:

```

exp_dir = get_experiment_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

pdf_name = get_report_name(
    NAMING_META['variante'],

```

```

    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'training',
    extras_id=extras_id
)

notebook_name = 'train_sae_matryoska-batch-topk.ipynb'
output_path = exp_dir / pdf_name

print(f'Generando PDF con Quarto...')
print(f'Archivo de salida: {output_path}')

# Quarto no acepta rutas en --output, solo nombre de archivo
# Se usa --output-dir para el directorio y --output solo para el nombre
result = subprocess.run(
    f'quarto render {notebook_name} --to pdf --output-dir "{exp_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)

if result.returncode == 0:
    print(f'PDF generado exitosamente: {output_path}')
else:
    print(f'Error al generar PDF:')
    print(result.stderr)

```